week04_keyword

▼ ES

ECMAScript는 정보 통신 시스템을 위한 국제 표준화 기구인 ECMA International에서 제정한 **자바스크립트의 표준 규격**이다. 자바스크립트는 언어이고, 그 언어가 어떻게 동작해야 하는지 정의한 설명서가 바로 ECMAScript라고 이해하면 된다. 브라우저마다 자바스크립트가 다르게 동작하는 것을 방지하기 위해 만들어졌다.

▼ ES6

2015년에 발표된 자바스크립트의 6번째 판본이다. 자바스크립트 역사상 가장 큰 변화가 있었던 버전으로, 현대적인 자바스크립트(Modern JavaScript)의 기점으로 평가받는다. 이전 버전(ES5)의 불편함을 대거 해소하고 언어의 완성도를 높였다.

▼ ES6의 주요 변화 및 특징

- **변수 선언 키워드(`let`, `const`):** 함수 레벨 스코프를 가진 `var` 대신 블록 레벨 스코프를 가진 변수 선언 방식이 도입되어 예측 가능한 코딩이 가능해졌다.

함수 레벨 스코프 vs 블록 레벨 스코프

스코프(Scope)는 변수가 유효한 범위를 말한다.
어떤 키워드로 변수를 선언하느냐에 따라 이 범위가 완전히 달라진다.

1. **함수 레벨 스코프 (Function-level Scope)**
   - **키워드:** `var`
   - **특징:** 오직 **함수 몸체 내부**만 지역 스코프로 인정한다.
   - **문제점:** 함수 내부에만 있다면 `if`, `for`, `while` 등 코드 블록(`{ }`) 안에서 선언해도 블록 밖에서 접근이 가능하다. 이는 의도치 않게 변수 값이 변경되는 버그의 주범이 된다.
   - **호이스팅:** 선언 전에도 참조가 가능(undefined)하여 코드의 흐름을 깨뜨릴 수 있다.
2. **블록 레벨 스코프 (Block-level Scope)**
   - **키워드:** `let`, `const` (ES6 도입)
   - **특징: 모든 코드 블록**을 지역 스코프로 인정한다. (함수, if문, for문, try/catch문 등) → 함수의 if문에서 선언한 count 라는 변수가 다음에 나오는 for 문에서의 count 변수랑 별개로 존재한다는 뜻.
   - **장점:** 변수의 생명 주기가 명확해지고 코드의 예측 가능성이 높아진다. 현재 현대적인 자바스크립트 개발에서는 `var`를 거의 사용하지 않고 이 방식을 표준으로 사용한다.

- **화살표 함수 (Arrow Functions):** `function` 키워드 대신 `=>`를 사용하여 함수를 간결하게 표현하며, 상위 스코프의 `this`를 그대로 계승한다.
- **템플릿 리터럴 (Template Literals):** 백틱(`` ` ``)을 사용해 문자열 안에 변수를 `${}`로 쉽게 삽입할 수 있게 되었다.
- **구조 분해 할당 (Destructuring Assignment):** 객체나 배열의 값을 분해하여 개별 변수에 즉시 할당할 수 있다.
- **클래스 (Classes):** 프로토타입 기반의 객체지향 프로그래밍을 다른 언어(Java 등)처럼 클래스 문법으로 직관적으로 작성할 수 있는 '문법적 설탕

(Syntactic sugar)'이 추가되었다.

- **프로미스 (Promises):** 비동기 처리를 위한 표준 객체가 도입되어 '콜백 지옥' 문제를 해결할 발판을 마련했다.

▼ ES6를 중요시 하는 이유

- **생산성 및 가독성 향상:** 문법이 간결해져 코드의 양이 줄고 가독성이 획기적으로 좋아졌다.
- **안전성 강화:** `let/const` 등을 통해 변수 오남용으로 인한 버그를 줄일 수 있다.
- **현대 프레임워크의 기반:** React, Vue, Angular 등 현대적인 프레임워크들이 ES6 이상의 문법을 기본으로 사용하기 때문에 필수적으로 익혀야 한다.

▼ ES Module

ESM 이라고 부른다. 자바스크립트 파일 자체를 하나의 모듈로 보고, 코드를 여러 파일로 쪼개서 관리할 수 있게 하는 표준 모듈 시스템.

- `export`: 변수, 함수, 클래스를 외부에서 사용할 수 있게 내보낸다.
- `import`: 외부 파일에서 내보낸 모듈을 가져와 사용한다.
- 이전에는 CommonJS(Node.js 방식) 등을 사용했지만, ES6부터는 **언어 차원에서 표준 모듈 시스템을 지원하게 되었다.**

언어차원에서 표준 모듈 시스템을 지원한다?
과거 자바스크립트는 '모듈'이라는 개념 자체가 없었다. 거대한 파일 하나에 코든 코드를 때려넣거나 여러 파일을 브라우저에 순서대로 로드 해서 썼다. (왜냐하면 자바스크립트는 원래 브라우저용 언어였기 때문이다. 웹 브라우저 않에서 짧은 스크립트를 돌리는 용도로만 쓴 것.)
그러다가 Node.js 가 처음 나왔을 때, 거대 서버 애플리케이션을 만들 수 있게 된 것. 서버 코드는 수만줄이 넘기 때문에 파일을 나누는 모듈 기능이 필요했다. 모듈 시스템이 절실했기 때문에 `require` , `module.exports` 이라는 키워드를 만들어서 임시방편으로 모듈 시스템을 제공했다.
그러나가 ES6가 발표되면서, `import/export` 기능(문법)이 자바스크립트 언어에 추가 된 것이다.

```
// 가져오기
import { add, PI } from './math.js';
import multi from './math.js';

console.log(add(5, 10)); // 15
console.log(PI);         // 3.14
console.log(multi(2, 3)); // 6
```

▼ TypeScript

자바스크립트에 **정적 타입** 시스템을 추가한 자바스크립트의 상위 집합언어다. 참고로, 마이크로소프트에서 개발했다.

구분	JavaScript (JS)	TypeScript (TS)
타입 결정	**동적 타이핑** (실행 시점에 결정)	**정적 타이핑** (컴파일 시점에 결정)
오류 발견	런타임(실행 중)에 오류 확인 가능	컴파일(실행 전) 단계에서 오류 발견
학습 곡선	낮음 (유연함)	높음 (엄격한 문법과 타입 정의 필요)
실행 환경	브라우저, Node.js에서 즉시 실행	JS로 변환(Transpile) 과정을 거쳐야 함

- **특징:** 코드를 실행하기 전(컴파일 단계)에 타입 오류를 잡아낼 수 있어 대규모 프로젝트의 유지보수에 매우 유리하다.
- **관계:** 결국 브라우저는 자바스크립트만 이해하므로, 작성한 TypeScript 코드는 자바스크립트로 변환(Transpile) 과정을 거쳐 실행된다.
- **보완된 점**

1. 강력한 에러 방지 (Type Safety)
JS는 숫자가 들어가야 할 곳에 문자열이 들어가도 실행 전에는 알 수 없다. TS는 코드를 짜는 도중에 타입을 체크하여 잘못된 데이터가 흐르는 것을 원천 차단한다.

2. 코드 가이드 및 자동 완성 (IDE Support)
객체에 어떤 속성이 있는지, 함수의 매개변수가 무엇인지 일일이 기억할 필요가 없다. VS Code 같은 에디터에서 강력한 자동 완성을 지원하므로 개발 속도가 오히려 빨라지고 오타로 인한 버그가 사라진다.

3. 협업과 유지보수 용이성
대규모 프로젝트에서 다른 사람이 짠 코드를 볼 때, 변수나 함수의 타입을 확인하는 것만으로도 어떤 의도로 작성되었는지 파악하기 쉽다. 일종의 '살아있는 문서' 역할을 한다.

4. 인터페이스(Interface)와 제네릭(Generic)
객체의 구조를 정의하는 `Interface`나 다양한 타입을 유연하게 처리하는 `Generic` 기능을 통해 훨씬 정교하고 재사용성 높은 객체지향 프로그래밍이 가능하다.

▼ 프로젝트 아키텍처

🍠 워크북에서 제안한 방식

- 거시적: modular monolith 방식 선택. 예를 들어, `modules/users`, `modules/payments` 식으로 폴더를 나눔
- 미시적: 각 모듈 내부에서는 ervice-Oriented Architecture(Service Layer Pattern)방식 선택.

▼ 프로젝트 아키텍처가 중요한 이유

1. 유지보수성 및 확장성
2. 코드 탐색 용이성 : 기능 중심으로 코드가 나뉘어 있어, 필요한 API 코드 찾는 시간이 단축됨.
3. 협업 효율 증대

▼ Modular Monolith Architecture

전체적인 구조를 잡을 때 사용하는 아키텍쳐. 즉, **프로젝트 전체를 어떻게 쪼갤까?**

비즈니스 도메인에 따라 **모듈을 분리**하되, **하나의 서버 단위에서 동작하는 구조.**

"하나이 서버 단위에서 동작하는 구조" → 이게 정확히 무슨 의미일까?
여러 기능을 쪼개놓긴 했지만, 결국 하나의 실행 파일로 묶어서 하나의 프로세스(Process)로 돌린다는 의미.

모놀리스 vs 마이크로서비스 (MSA)

- **모듈형 모놀리스 (Modular Monolith):** 코드는 '유저', '결제' 등 도메인별로 엄격하게 나눠놓지만(모듈화), 실제 실행할 때는 하나의 프로그램(`.exe`나 `node index.js`) 안에서 다 같이 돌아가는 구조.
- **마이크로서비스 (Microservices Architecture, MSA):** '유저 서버', '결제 서버', '게시판 서버'를 **물리적으로 아예 다른 컴퓨터(또는 프로세스)**에 띄우는 것. 서버끼리 통신하려면 네트워크를 타야 한다는게 차이점. 관리는 힘들지만, 결제 서버가 터져도 로그인 서버는 살아있다는 장점이 있다.

- 레이어 중심이 아닌, 기능(결제, 사용자, 알림) 중심으로 모듈을 나눈다.
- 장점: 모듈 간 접근을 정해진 인터페이스나 공개 API를 통해서만 이루어지도록 제한하여, 특정 기능을 수정할 때 시스템 전체에 미치는 영향 범위를 명확히 통제할 수 있다.

▼ Service-Oriented Architecture(Service Layer Pattern)

내부 설계 → 쪼갠 모듈 안에서 코드를 어떻게 layer 별로 나눌 것인가에 대한 방법. 비즈니스 로직을 Controller와 분리하여 Service 계층에 몰아넣는 것이 핵심.

모듈 내부에서 비즈니스 로직을 분리하기 위해 사용하는 레이어드 아키텍처의 한 형태다.

- **Controller (컨트롤러):** 클라이언트의 요청을 받아 서비스에 전달하고 응답을 반환하는 입구 역할을 한다.
- **Service Layer (서비스 레이어):** 실제 비즈니스 로직이 구현되는 핵심 계층으로, 요청을 처리하고 데이터를 가공한다.
- **Data Access / Repository Layer:** 데이터베이스와 직접 소통하며 쿼리를 수행하는 계층이다.

▼ MVC 패턴

Model, View, Controller로 구성

- **Model:** 데이터와 비즈니스 로직을 담당한다.

- **View:** 사용자에게 보여지는 화면을 처리한다 (API 서버의 경우 JSON 응답이 이 역할을 대신한다).
- **Controller:** 사용자 입력을 받아 모델을 업데이트하거나 뷰를 선택한다.

▼ 그 외 다른 프로젝트 구조

- **단순 레이어드 아키텍처:** 모듈화 없이 단순히 Controller-Service-Repository 레이어만 나누어 코드를 작성하는 방식이다.
- **Microservices Architecture (MSA):** 각 모듈을 별도의 서버로 분리하여 운영하는 구조로, 대규모 시스템 확장에 유리하다.

▼ 비즈니스 로직

> 프로그램이 돌아가는 핵심 규칙

역할: 컨트롤러(Controller)로부터 전달받은 요청을 실제로 처리하는 계층이다. 단순히 데이터를 주고받는 게 아니라, "할인 쿠폰은 1인당 1매만 적용 가능하다" 같은 **서비스의 정책과 규칙**이 여기서 실행된다.

▼ DTO

Data Transfer Object → 데이터 전송 객체...

- **역할:** 계층(Layer) 사이에서 데이터를 전송할 때 사용하며, 특히 클라이언트가 보낸 데이터를 객체로 변환할 때 쓰인다.
- DTO는 단순히 데이터를 묶어주는 역할뿐만 아니라, 서버의 입구에서 **데이터의 무결성을 보장하는 방어막** 역할도 한다.
- **사용 이유:**
  - **유효성 검증:** 값이 빠졌거나 형식(이메일 등)이 틀렸는지 미리 확인한다.
  - **데이터 필터링:** DB에서 가져온 정보 중 패스워드 같은 민감한 정보는 빼고 필요한 것만 컨트롤러로 보낼 수 있다.
  - **통일성:** 파라미터가 너무 많아질 때 하나로 묶어 전달하면 코드가 깔끔해지고 유지보수가 쉬워진다.

실제 코드에서 DTO의 구현 모습

```
// AddWishListRequestDto 예시 코드
export class AddWishListRequestDto {
    // 데코레이터(@)를 이용해서 자동 검증을 수행한다. c
lass validator라이브러리 기능
  @IsNotEmpty() // 필수 값 체크
  @IsString()   // 문자열 타입 체크
  productName!: string;

  @IsNotEmpty()
  @IsNumber()   // 숫자 타입 체크
  price!: number;

  @IsUrl()      // URL 형식 체크
  @IsString()
  url!: string;

  @IsNotEmpty()
  @IsString()
  storeName!: string;

  @IsNotEmpty()
  @IsString()
  brandName!: string;

  @IsString()
  reason!: string;

  @IsString()
  userId!: string;

  photoFile?: Express.Multer.File; // 파일 첨부
(선택 사항)
  // 생성자(constructor)를 통해 필요한 데이터만 골라
담는다.
  constructor(data: {
    userId: string;
```

```
    productName: string;
    price: number;
    url: string;
    storeName: string;
    brandName: string;
    reason: string;
    photoFile?: Express.Multer.File;
  }) {
    this.userId = data.userId;
    this.productName = data.productName;
    this.price = data.price;
    this.url = data.url;
    this.storeName = data.storeName;
    this.brandName = data.brandName;
    this.reason = data.reason;
    this.photoFile = data.photoFile;
  }
}
```